

PROJECT REPORT

Course Code: CSE1021

VEHICLE RENTAL MANAGEMENT SYSTEM

Submitted by:

Name : MAYANK MOHAN

Registration No. : 26BAI11096

Programming Language : Python

Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Expected Console Output
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

2. Introduction

This project is a Vehicle Rental Management System written in Python. It runs in the console and is meant to help a rental operator handle the day-to-day work of leasing out vehicles. With it, the operator can keep track of which vehicles are in the fleet, rent them out to customers, take them back when they are returned, look through old records, and see how much money has been earned in total. I kept the design simple so that anyone can use it just by following the menu.

3. Problem Statement

When a rental fleet is managed by hand, a few problems come up again and again:

- It is easy to lose track of which vehicles are actually available.
- Rent calculations done by hand for the number of days can go wrong.
- Customer rental history is hard to maintain, and it is difficult to check how much income has been made.
- There is no live status, so a vehicle can get double-booked or a return can be missed.

This system tackles these problems by updating vehicle status automatically, calculating the rent on its own, allowing searches, and keeping a log of every transaction.

4. Functional Requirements

Area	What the system should do
Inventory Display	Show only the available vehicles (show_vehicles()), and show the whole fleet along with each vehicle's status (show_all_vehicles()).
Search	Search for a vehicle by its full or partial name, ignoring upper/lower case (search_vehicle()).
Renting	Book a vehicle after checking it is free (an already rented vehicle cannot be rented again). Take the customer's name, phone number and number of days, work out the total rent, and change the vehicle's status to "Rented".
Returning	List the vehicles that are currently rented and let the operator return one (return_vehicle()), which sets its status back to "Available".
Records and Income	Show the full history of completed rentals (show_rental_records()) and add up the total income from all of them (total_income()).

5. Non-functional Requirements

Requirement	Details
Usability	A simple menu-driven console interface.
Reliability	Invalid input, such as text where a number is expected, is handled with try-except (ValueError) so the program does not crash.
Performance	Updates happen instantly. Searching the vehicles and going through the records takes $O(N)$ time.
Maintainability	The code is split into functions, which keeps it organised and easy to change.

6. System Architecture

The program follows a modular, procedural structure. Everything is controlled by one main menu loop (`while True`) that calls the required function based on the user's choice.

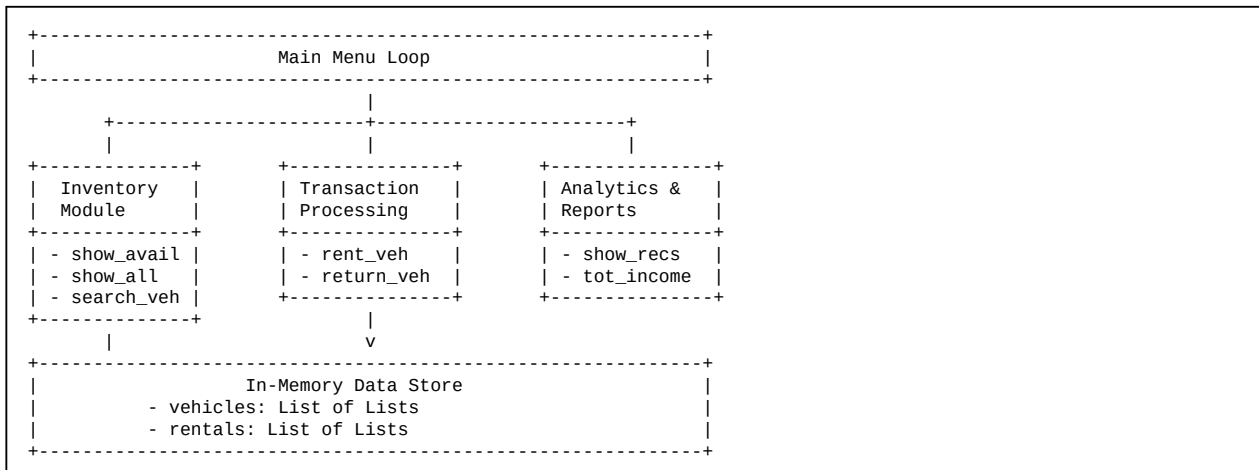
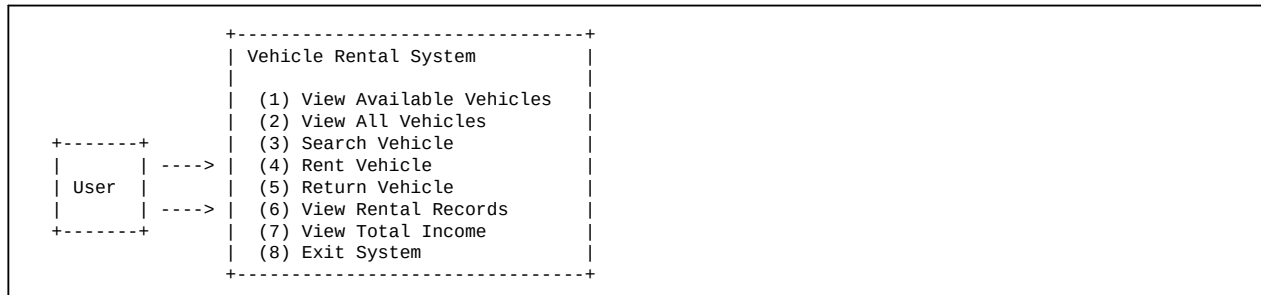


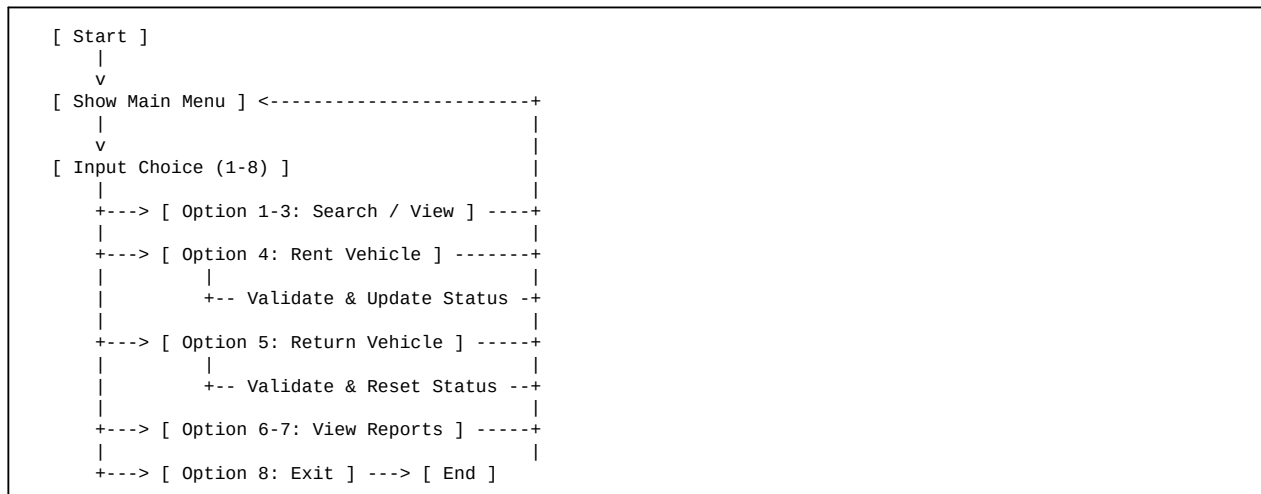
Figure 1: System architecture

7. Design Diagrams

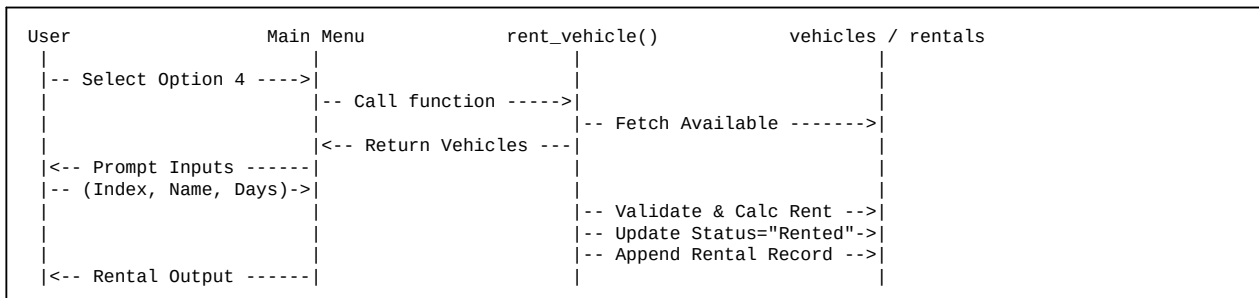
Use Case Diagram



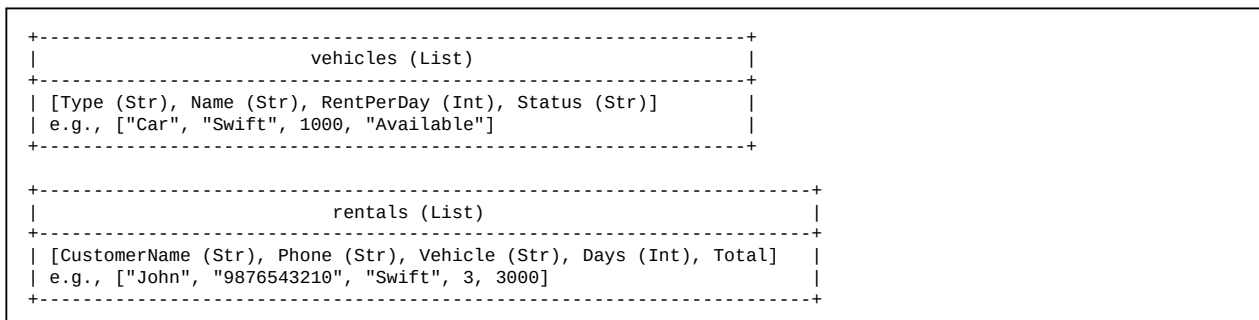
Workflow Diagram



Sequence Diagram (Rent Vehicle)



Component / Data Structure Diagram



8. Design Decisions and Rationale

Decision	Why I chose it
Nested lists for data	They are simple, need no external libraries, and use very little memory.
Status flags	The strings "Available" and "Rented" show the state of each vehicle, which stops overbooking and duplicate returns.
Substring matching	Using <code>.lower()</code> together with the <code>in</code> operator makes the search case-insensitive.
Input validation	Numbers are checked with simple conditions (<code>number < 1</code> or <code>number > len(vehicles)</code>) along with try-except blocks.

9. Implementation Details

The project is written in Python 3.x. The main functions are:

Function	What it does
show_vehicles()	Goes through the fleet and picks vehicles whose status (index 3) is "Available".
show_all_vehicles()	Displays the complete list, whatever the status.
search_vehicle()	Searches the vehicle name (index 1) for the text entered.
rent_vehicle()	Takes the inputs, checks availability, calculates the total cost, updates the status and adds the booking to the rentals list.
return_vehicle()	Lists rented vehicles and sets the chosen one back to "Available".
show_rental_records() and total_income()	Go through the saved bookings to show the transaction log and add up the total income.

10. Screenshots / Expected Console Output

Main Menu

```
=====
      VEHICLE RENTAL MANAGEMENT SYSTEM
=====
1. Show Available Vehicles
2. Show All Vehicles
3. Search Vehicle
4. Rent Vehicle
5. Return Vehicle
6. Show Rental Records
7. Show Total Income
8. EXit
=====
Enter your choice: 1
```

Renting a Vehicle

```
===== AVAILABLE VEHICLES =====
1 | Car | Swift | Rs. 1000 per day
2 | Bike | Acura | Rs. 500 per day

Enter vehicle number: 1
Enter customer name: Alex
Enter phone number: 9876543210
Enter number of days: 2

===== RENTAL SUCCESSFUL =====
Customer: Alex
Phone: 9876543210
Vehicle: Swift
Rent per Day: Rs. 1000
Days: 2
Total Rent: Rs. 2000
Status: Rented
=====
```

11. Testing Approach

Test ID	Feature	Input	Expected Result	Result
TC-01	Show Available Vehicles	Choice 1	Displays list of unrented vehicles	PASS
TC-02	Search Existing Vehicle	Search term "swift"	Displays details of Swift	PASS
TC-03	Rent Vehicle Validation	Rent an already rented vehicle	Displays "This vehicle is already rented."	PASS
TC-04	Invalid Days Input	Days = 0 or -2	Displays "Days should be greater than 0."	PASS
TC-05	Non-integer Input	Choice/Index "abc"	Catches ValueError gracefully	PASS

12. Challenges Faced

- **Keeping the state correct:** I had to make sure renting or returning a vehicle changes its status properly without creating duplicate entries.
- **Crashes on wrong input:** The program used to crash when a non-number was typed. I fixed this by putting try-except blocks around the integer inputs.
- **Index errors:** Users pick vehicles starting from 1, but Python lists start from 0, so I subtracted 1 (`number - 1`) to match them.

13. Learnings and Key Takeaways

- I got a practical understanding of modular programming using functions in Python.
- I gained hands-on experience with control flow: while loops, if-elif-else and for loops.
- I learned how to handle user input and stop it from causing runtime errors.

14. Future Enhancements

- **Permanent storage:** Use files (JSON/CSV) or an SQLite database so records are not lost when the program is closed.
- **GUI or web version:** Build a graphical interface with Tkinter or PySide, or turn it into a web application with Flask.
- **More features:** Add date and time stamps, seasonal pricing for peak periods, and customer identity verification.

15. References

- Python Documentation: Built-in Functions and Data Structures (docs.python.org)
- IEEE Software Engineering Guidelines for Project Documentation.